UNIVERSIDAD SIGLO 21

Seminario Práctico de Informática

Módulo 1

Trabajo Práctico 3



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen
DNI: 44933871
Profesor Titular: Pablo Alejandro Virgolini
Carrera: Licenciatura en Informática
Año: 2026 - 2.° cuatrimestre
Fecha de entrega: 17 de octubre de 2026

Índice

1. Introducción

2. Lo que cambió respecto del TP2 (y por qué)

3. Correcciones aplicadas al TP1

4. Correcciones aplicadas al TP2

5. El prototipo: arquitectura y estructura del código

6. Decisiones de diseño que vale la pena contar

7. Persistencia: lo que aprendí escribiendo el DAO

8. Colecciones, algoritmos y el cronómetro en vivo

9. El TestRunner: 14 casos y la pequeña enseñanza del CP12

10. Base de datos: el script de integridad

11. Compilación, ejecución y el fallback en memoria

12. Cierre: lo que me llevo de este TP

13. Referencias bibliográficas

Anexo A. Diagramas UML del proyecto

1. Introducción

Este es el tercer y último entregable del AP1 de Práctica Solidaria y, como es la tercera vez que escribo sobre ReperTrack, voy a intentar no repetir lo que ya está en el TP1 y el TP2. Quien esté leyendo esto en orden probablemente ya vio la justificación del problema, el análisis del modelo de dominio y la primera versión del prototipo con su base de datos. Voy a dar por sentado todo eso y enfocarme en lo que efectivamente cambió en esta entrega: las correcciones que me quedaron pendientes, la decisión fuerte de pasar todo el frontend a consola pura, y algunas cosas que aprendí en el camino mientras escribía el código.

Si tuviera que resumir este TP en una frase, diría que se trata de dejar ReperTrack andando de verdad, no sólo dibujado en un diagrama. Cuando arranqué el TP1 todavía no había escrito una sola línea de Java del proyecto, y para cuando entregué el TP2 ya tenía un prototipo con Swing funcionando pero todavía con varios cabos sueltos (la vista estaba amarrada a mano, las validaciones se disparaban en lugares distintos, la planificación de los casos de uso no estaba 100% cerrada). Este TP3 cierra eso: consola, JDBC transaccional, excepciones personalizadas, y un TestRunner que verifica catorce casos y los deja en verde.

También aprovecho para dejar la recomendación obvia que leí muchas veces en los libros de la carrera pero recién ahora entendí escribiendo: en un prototipo chiquito como éste, la decisión más importante no es el patrón arquitectónico (MVC, capas, repositorio), sino cuidar los límites de los objetos. Cuando uno deja que la sesión sepa del instrumento por un int pelado en lugar de una asociación, todo el resto del código empieza a mentir un poquito. De ahí la importancia de la corrección que apliqué en el modelo de TP2.

2. Lo que cambió respecto del TP2 (y por qué)

Lo más visible: el frontend dejó de ser Swing y pasó a ser una aplicación de consola con menú de texto. La consigna del TP3 era explícita al respecto, así que la pregunta no era si migraba sino cómo lo hacía sin perder lo que ya estaba.

La parte que me llevó más tiempo fue decidir dónde vivía la antigua VistaSesion. La opción obvia era borrarla, pero eso me dejaba sin trazabilidad histórica de la implementación Swing que figuraba en el TP2. Terminé moviéndola a un paquete com.repertrack.legacy con la clase marcada `@Deprecated` y un comentario bien grande arriba explicando que está ahí sólo como referencia. Así, si alguien en una iteración futura quiere mostrarle a un profesor cómo era la GUI, sigue estando en el repositorio, pero no participa del classpath principal y nadie la ejecuta por accidente.

Lo otro que reescribí desde cero fue el modelo. En el TP2, SesionDePractica tenía instrumentoId y elementoId como atributos sueltos, y lo mismo hacía ElementoDeEstudio con su instrumentoId. Esa representación andaba, pero a la hora de escribir los DAOs y los servicios se notaba que faltaba algo: no podía preguntarle a una sesión cuál era su instrumento, sólo podía pedirle el id e ir a buscarlo. El TP3 cambia eso a asociaciones reales: SesionDePractica tiene un Instrumento y un ElementoDeEstudio. Lo mismo con el resto del modelo. Eso obligó a actualizar constructores, DAOs y servicios, pero el código quedó bastante más limpio.

3. Correcciones aplicadas al TP1

3.1. Objetivos reescritos

El TP1 tenía unos objetivos específicos bastante largos, mezclando el objetivo en sí mismo con sus criterios de aceptación. Los reescribí aplicando la regla que más me sirvió de las lecturas de la materia: un verbo en infinitivo, una idea, no más de dos líneas, y verificable al cierre de la iteración

del PUD. Quedaron seis objetivos específicos en lugar del listado original, todos verificables (uno mira la base, otro mira la UI, otro mira el repositorio, etc.).

También separé claramente el Objetivo General del Proyecto (lo que se entrega al cliente) del Objetivo General del Sistema (lo que el sistema hace día a día). En el TP1 estaban casi iguales, y eso es un error conceptual que confunde cuando uno después quiere derivar métricas de aceptación.

3.2. Diagrama de dominio ajustado

El diagrama original omitía los atributos de cada clase y, peor todavía, no mostraba la asociación directa Músico --registra-- SesiónDePráctica. Esto último era una molestia porque, sin esa arista, el modelo daba a entender que las sesiones estaban sueltas en el sistema. Lo arreglé agregando la asociación y dejando explicitada la tabla de clases como texto alternativo al diagrama, así no se pierde nada cuando se imprime en blanco y negro.

3.3. Propuesta técnica migrada a consola

Éste fue el cambio más costoso del TP1 hacia el TP3, porque la propuesta técnica del TP1 mencionaba Swing y JavaFX en cinco lugares distintos: en las T.I.C, en la sección 8.2 de propuesta técnica, en RNF01 y RNF06, y dentro del curso normal de CU006. Tuve que ir corrigiendo uno por uno. Lo que aprendí en el proceso es que cambiar la capa de presentación no es sólo cambiar la vista: hay que asegurarse de que el resto del documento no quede diciendo cosas que ya no son verdad. Por eso actualicé el CU006 también, reemplazando diálogo modal Swing por prompt por consola y el cronómetro de `javax.swing.Timer` a `System.currentTimeMillis()` con un bucle simple.

4. Correcciones aplicadas al TP2

4.1. Diagrama de clases de diseño (visibilidades, tipos, asociaciones)

El diagrama de clases del TP2 ya estaba bastante armado, pero le faltaba explicitar visibilidades (+, -, #), los tipos Java de cada atributo, y las firmas de los métodos. También tenía una cosa rara: en el texto se decía que ElementoDeEstudio tenía una asociación con Instrumento, pero el diagrama lo dibujaba como un instrumentoId. Esa contradicción interna del modelo la corregí explicitando la asociación como tal, con su rol y su cardinalidad. El resto de las clases quedaron más completas a nivel de tipado, lo que ayuda a leer el diagrama sin tener que abrir el código.

4.2. Secuencia de diseño usando las clases de diseño

Esta corrección me resultó media tonta cuando la entendí, pero es importante: el diagrama de secuencia del TP2 mostraba una clase VistaSesion que no estaba en el diagrama de clases, y omitía los controladores que tampoco estaban ahí. O sea, la secuencia referenciaba clases que no existían en ningún otro lado del documento. Lo que hice fue renombrar las clases de controlador con nombres concretos (MenuPrincipal, SesionController, etc.), incluirlas en el diagrama de clases de diseño, y reescribir la secuencia para que use exactamente esos nombres. Así el diagrama de secuencia y el diagrama de clases dejan de ser dos documentos que se contradicen y pasan a ser el mismo modelo contado de dos maneras.

4.3. Diagrama de despliegue y aclaración del CU011

El despliegue del TP2 ya estaba bien enfocado (PC local con MySQL, sin servidor web), pero le faltaban los artefactos (no decía cuál era el .jar, ni el archivo SQL). Lo completé incluyendo ReperTrack-app.jar, el esquema SQL y el script de pruebas de integridad como artefactos separados.

El CU011 estaba mencionado sólo de pasada en el TP2, sin especificación. Lo redacté con el mismo template que usé para CU006: precondiciones, postcondiciones, curso normal, alternativas, excepciones y reglas de negocio. La RN-07 (rechazar exportación si no hay datos) sale explícitamente del TestRunner en CP14.

4.4. Plan de pruebas unificado (CP01..CP14)

Ésta es una corrección que duele un poco porque en el TP2 la nómina del plan no cerraba. El plan tenía cinco casos rotulados CPC01, CPF02, CPS03, CPS04 y CPS05. Los resultados de ejecución listaban catorce casos: CP01, CP02, ..., CP14. Es decir, no se podía saber que CPC01 era lo mismo que CP01, ni cuántos casos conformaban realmente el plan. Lo dejé todo unificado bajo la nómina CP01..CP14, con su nivel, su técnica y qué verifica cada uno, y armé una tabla aparte con la entrada y el resultado esperado de los catorce. Cuando uno mira el TestRunner nuevo y la tabla, puede rastrear uno a uno sin ambigüedad.

4.5. Prototipos de interfaz de consola

El TP1 hablaba de una UI Swing y el TP2 mostraba una vista con JFrame. Como ahora la UI es consola, dibujé seis prototipos en ASCII con bordes: Menú Principal, Submenú Instrumentos, Submenú Elementos, Registrar Sesión (con el cronómetro en vivo), Estadísticas y Alertas. Los prototipos no son vinculantes: la implementación real es un poco distinta porque algunas opciones las agregué mientras escribía el código y me parecieron útiles (por ejemplo, el submenú de deshacer en Elementos, que nació de la pila que armé para el requisito de colecciones).

4.6. DER con tipos SQL y script de integridad en MySQL

El TP2 dibujaba el DER sin tipos SQL por atributo (uno tenía que abrir el script para saber si musico_id era INT o BIGINT), y además las pruebas se habían corrido en SQLite. Lo reescribí como puro MySQL 8: con `ENGINE=InnoDB`, `utf8mb4_unicode_ci`, `ENUM` para los tipos cerrados, `CHECK` enforcement en cada regla de negocio, y un script separado (05_pruebas_integridad.sql) con diez pruebas que verifican FK, UNIQUE y CHECK contra un MySQL real. Lo aclaro en el informe porque SQLite no enforza CHECK antes de la versión 3.32 y no soporta CASCADE, así que correr las pruebas allí daba una falsa sensación de seguridad.

5. El prototipo: arquitectura y estructura del código

La estructura final respeta el patrón MVC declarado en el RNF03 del TP1, pero adaptada a consola pura. Los paquetes quedaron así:

```
src/com/repertrack/
  modelo/         Musico, Instrumento, ElementoDeEstudio (abstract),
                  Cancion, Ejercicio, SesionDePractica,
                  Estadistica, Alerta + 4 enums (Nivel, TipoInstrumento,
                  TipoEjercicio, CategoriaElemento)
  excepcion/      6 excepciones checked
  util/           ConexionDB + Validador
  dao/            InstrumentoDAO, ElementoDAO, SesionDAO + 3 impls
  service/        InstrumentoService, ElementoService, SesionService,
                  EstadisticaService, MotorAlertasService,
                  ExportadorPdfService
  controlador/    MenuPrincipal + 5 subcontroladores de consola
  estructuras/    ListaEnlazadaSimple, Pila, Cola, Busqueda, Ordenacion
  legacy/         VistaSesion (@Deprecated, referencia del TP2)
```

El Main arranca el menú principal y se cierra al elegir la opción 0. Los subcontroladores leen con Scanner y escriben con `System.out.printf`, con tablas formateadas en ASCII. El SesionController tiene un cronómetro en vivo basado en `System.currentTimeMillis()`: el usuario teclea `I` para iniciar, `P` para pausar y `F` para finalizar, todo dentro de un `while` que va sumando intervalos. La conversión a minutos se hace al final con un `max(1, segundos/60)` para asegurar que la sesión cumpla RN-02 incluso si el usuario fue muy rápido.

Un detalle que vale la pena contar: el SesionController mantiene una `ListaEnlazadaSimple<SesionDePractica>` con las sesiones registradas en la corrida actual. La uso para mostrar el historial inmediato sin volver a pegarle a MySQL. Esa estructura de datos es una de las que la consigna del TP3 pedía, así que cumple doble función: requisito académico y optimización real.

6. Decisiones de diseño que vale la pena contar

6.1. Single-table inheritance en elemento_estudio

Una discusión que tuve conmigo mismo fue si la herencia Canción/Ejercicio tenía que reflejarse en la base con tres tablas (una por jerarquía, una por clase, etc.) o con una sola. Opté por single-table por dos razones concretas. La primera: el caso de uso que más se ejecuta (registrar una sesión) siempre necesita "el elemento", sea canción o ejercicio, así que con una tabla es un solo JOIN en lugar de dos con UNION. La segunda: las alertas de mantenimiento barren todos los elementos activos sin importar tipo, y con tres tablas serían tres SELECT en lugar de uno solo. El precio (algunas columnas NULL según la categoría) lo controlo con dos CHECK: `chk_elem_cancion` exige que Canción tenga autor y Ejercicio tenga tipo_ejercicio, así que la base se encarga sola de mantener la invariante del dominio.

6.2. Polimorfismo sobre calcularProgreso(int)

Éste es el lugar donde el modelo justifica la herencia en términos de código. Una canción difícil (digamos, dificultad 8) pide muchas más horas de práctica que un ejercicio difícil 8, porque el objetivo es estético y depende de pulir detalles; un ejercicio difícil 8 pide un volumen alto pero es más mecánico. Por eso Canción divide por `dificultad * 120` y Ejercicio divide por `dificultad * 60`. El polimorfismo se manifiesta en que `SesionService.progresoElemento(elemento, minutos)` invoca el método sobre la referencia del tipo padre y la JVM despacha la versión concreta. Esto está verificado por CP08 y CP09.

6.3. Encapsulamiento + excepciones checked

Todos los atributos del modelo son `private` y el acceso se hace por getters/setters. Los setters no son pasivos: cada uno valida con la clase `Validador` y lanza una excepción checked si el dato no cumple. Esto fuerza a quien llama a manejar el error explícitamente: no se puede instanciar una Canción con dificultad 11 sin atrapar (o declarar) la `RangoInvalidoException`. La regla la verifiqué con CP06 y CP07, que son los que arman un elemento con valores inválidos y esperan el rechazo.

6.4. Transacciones ACID en el DAO

Para cumplir con el RNF05 (atomicidad transaccional) uso el patrón clásico: `setAutoCommit(false)` al inicio, `commit()` si todo salió bien, `rollback()` en cualquier `SQLException`. Lo importante es que el rollback se hace dentro del `catch` para garantizar que, si la FK del elemento no existe, la sesión a medio insertar no quede colgando. Todo el código está dentro de un `try-with-resources`, así que la Connection se cierra siempre, incluso si falla el rollback (en cuyo caso se pierde la transacción pero el programa sigue corriendo, que es lo importante para un prototipo).

7. Persistencia: lo que aprendí escribiendo el DAO

Lo primero que decidí cuando me senté a escribir los DAOs fue que iban a tener un fallback automático a memoria si MySQL no estaba disponible. La idea es que el prototipo pueda demostrar el flujo end-to-end aunque el alumno (o el corrector) no tenga MySQL levantado en la máquina. Para eso, el constructor del DAO chequea `ConexionDB.mysqlDisponible()`: si falla, setea una bandera `enMemoria = true` y todos los métodos escriben/leen sobre un `ArrayList` interno. Si no falla, el camino es JDBC puro.

El DAO de elementos es el más interesante porque tiene que decidir, leyendo la columna `categoria` del ResultSet, si la fila corresponde a una Canción o a un Ejercicio. Eso lo hago con un `if-else` sobre el valor de la columna y construyo la subclase concreta con todos sus setters. El `else` mapea los campos del Ejercicio (incluido el BPM, con `rs.wasNull()` para distinguir entre un BPM 0 y un BPM NULL que la base no rechazó por el CHECK).

El SesionDAO es el más directo: solo dos foreign keys, la fecha y el tiempo. Pero el método `guardar()` devuelve el id generado y, en el camino JDBC, lo hace con `RETURN_GENERATED_KEYS`. Esto me obligó a lidiar con un detalle que parece tonto: si el INSERT no afectó ninguna fila (por ejemplo, porque la sesión ya existía por un id duplicado en la prueba), no hay claves generadas y tengo que devolver 0 sin que la JVM se queje. Nada raro, pero me llevó unos minutos averiguarlo.

8. Colecciones, algoritmos y el cronómetro en vivo

La consigna del TP3 pedía explícitamente el uso de alguna estructura de datos (pila, cola o lista enlazada) y, si se justificaba, algún algoritmo de ordenación o búsqueda. Decidí implementar las tres estructuras a puntero, sin librerías externas, porque me parecía más didáctico que esconderlas detrás de un `java.util.ArrayDeque`. Cada una tiene un test natural dentro del sistema:

`ListaEnlazadaSimple<T>` la usa el SesionController para mantener el historial reciente de sesiones dentro de la corrida. La iteración es con un `for-each`, lo cual implica que la lista implementa Iterable. Eso me obligó a escribir un Iterator manual con los métodos `hasNext()` y `next()`.

`Pila<T>` la usa el ElementoService para soportar la opción 5 del submenú Elementos ("Deshacer último alta"). Cada vez que el usuario da de alta un elemento, se apila; si se arrepiente, lo desapila. En la implementación actual no elimino el elemento de la base (eso sería un rollback, mucho más complejo), pero la pila está ahí como requisito académico y para una futura mejora. La idea es que cuando alguien en una iteración posterior quiera hacer un verdadero rollback, la mecánica ya está probada.

`Cola<T>` la usa el MotorAlertasService para despachar las alertas en orden FIFO. Cada elemento activo que supere los N días sin práctica encola una `Alerta`; el AlertaController drena la cola con un `while (!cola.estaVacia()) cola.desencolar()` y va imprimiendo. Es el comportamiento más natural para una cola.

Para algoritmos, escribí dos de ordenación (selección y burbuja) y dos de búsqueda (lineal y linealIndice). Los métodos son genéricos: `seleccion(List<T>, Comparator<T>)` y `burbuja(List<T>, Comparator<T>)`. La diferencia principal es que selección hace menos intercambios en promedio y burbuja sale antes en listas casi ordenadas. En el SesionService los uso concretamente: burbuja para ordenar por tiempo descendente (es la opción que más usa el contador "mostrame las sesiones más largas primero") y selección para ordenar por fecha descendente (que es lo que ve el usuario cuando hace "Listar sesiones").

El cronómetro en vivo no es una estructura de datos pero vale la pena contarlo porque es donde se ve mejor el flujo de control. El `SesionController.cronometrar()` mantiene `acumuladoMs` y `pausaInicio` como `long`. Las teclas `I`, `P` y `F` modifican el estado. La conversión final es `max(1, segundos/60)` para respetar RN-02 incluso en sesiones muy cortas. Si el usuario cierra el programa a mitad del cronómetro, la sesión se pierde (no hay autoguardado), pero eso está bien porque el prototipo es monousuario y asumo que el usuario no apaga la máquina a mitad de un ensayo.

9. El TestRunner: 14 casos y la pequeña enseñanza del CP12

El TestRunner es un reemplazo chico de JUnit, hecho a mano con un método `check(boolean, String)` que cuenta PASS y FAIL. Cuando ejecuto el comando del README me imprime 14 líneas PASS y un resumen final con 14 PASS, 0 FAIL. Esto es el criterio de 20 puntos de la rúbrica, así que es importante que verifique.

Los catorce casos se distribuyen así: CP01 a CP05 prueban SesionService y SesionDePractica (camino feliz, RN-01, RN-02, frontera, RN-04). CP06 y CP07 prueban ElementoService con valores inválidos de dificultad y BPM. CP08 y CP09 prueban el polimorfismo: Canción y Ejercicio calculan progreso distinto para los mismos minutos y dificultad. CP10 y CP11 prueban las estadísticas. CP12 prueba la alerta de 45 días sin práctica. CP13 y CP14 prueban la exportación a PDF: uno con datos, el otro sin datos debe fallar.

El caso que me llevó más tiempo fue el CP12, y la enseñanza es tan chica que la quiero contar. Inicialmente pasaba la sesión con fecha de hace 45 días al DAO y esperaba la alerta, pero no se disparaba. Después de un rato descubrí el bug: el DAO tomaba el `LocalDateTime.now()` al construir la sesión y mi llamada a `setFechaRegistro()` se ejecutaba después del constructor, así que el orden de operaciones estaba al revés. La solución fue cambiar el constructor para que reciba la fecha como parámetro (en lugar de generarla internamente) y el TestRunner ahora le pasa la fecha vieja directamente. Es un ejemplo de libro de por qué los constructores no deberían tener side effects que uno no espera.

El otro detalle que vale la pena mencionar es que el TestRunner usa un `SesionDAOStub` (una clase interna que implementa `SesionDAO` y guarda en memoria) en lugar del DAO real. Esto desacopla las pruebas de la disponibilidad de MySQL: puedo correr el TestRunner desde cualquier máquina, sin tener que montar el esquema antes. La idea es la misma que usaría Mockito pero, como tampoco tengo Mockito disponible en este entorno, me lo escribí a mano. Tiene sus limitaciones (no se puede inyectar fallos simulados, por ejemplo), pero para los 14 casos que necesito es más que suficiente.

10. Base de datos: el script de integridad

El script `05_pruebas_integridad.sql` es una de las cosas que más me enorgullece de esta entrega, porque cierra el círculo de la rúbrica del TP2 (que mencionaba 10 pruebas de integridad) y del TP3 (que pedía scripts puros MySQL). Contiene diez INSERT/DELETE que prueban: tres CHECK (tiempo cero, dificultad fuera de rango, BPM fuera de rango), una FK inexistente, un UNIQUE duplicado, un DELETE con FK activa, un CHECK de comentario de 501 caracteres, un INSERT válido (smoke test positivo), un conteo de verificación, y la limpieza final.

Lo importante, y lo aclaro en el script porque se presta a confusión, es que esto no se puede correr contra SQLite. La razón es técnica: SQLite no enforza todas las restricciones CHECK antes de la versión 3.32, no soporta `ENGINE=InnoDB`, y la implementación de `ON UPDATE/DELETE CASCADE` no es la misma que en MySQL. Por eso, si alguien intenta correr el script en SQLite, los CHECK van a pasar silenciosamente y va a creer que sus datos son válidos cuando no lo son. La

única manera honesta de probar la integridad de ReperTrack es contra un MySQL 8 real.

La estructura final del esquema es la siguiente: cuatro tablas (musico, instrumento, elemento_estudio, sesion_practica), tres índices compuestos para acelerar las consultas de estadísticas, todas las FK con `ON DELETE RESTRICT` (no quiero perder sesiones si se borra un instrumento por accidente), y todas las reglas de negocio materializadas como CHECK. La normalización está verificada en 3FN: no hay dependencias parciales (las PK son subrogadas), no hay dependencias transitivas (el nombre del músico no vive en sesion_practica, se llega por instrumento), y los atributos son atómicos (el comentario es un único texto, no una lista).

11. Compilación, ejecución y el fallback en memoria

Para compilar uso el compilador ECJ 3.37 con nivel 11, que es lo que estaba disponible en mi máquina. El comando es `java -jar ecj.jar -source 11 -target 11 -d build` con todos los .java. Compila limpio: 0 errores y sólo warnings de `serialVersionUID` en las excepciones y de imports no usados. Cero ruido importante.

Para correr el Main se usa `java -cp build:mysql-connector-j-8.3.0.jar com.repertrack.Main`. Arranca con "Bienvenido a ReperTrack v1.0", carga el menú principal y se queda esperando una opción. Si MySQL no está levantado, los DAOs imprimen `[WARN] MySQL no disponible` y siguen funcionando con una lista interna. Esto es importante para la demo: aunque el evaluador no tenga MySQL, el sistema sigue siendo operacional.

El proyecto también incluye un `pom.xml` con la dependencia de mysql-connector-j 8.3.0, así que si el corrector prefiere usar Maven puede correr `mvn clean package` y todo cae en `target/repertrack-app.jar`. El Main-Class está declarado en el manifest, así que con un `java -jar target/repertrack-app.jar` ya funciona.

12. Cierre: lo que me llevo de este TP

Este TP me deja dos cosas que van más allá de la nota. La primera, más concreta: un prototipo que se compila, se ejecuta, pasa sus 14 pruebas y tiene los criterios de la rúbrica cubiertos. La segunda, más metodológica: la convicción de que en sistemas chicos como éste, la inversión más rentable no es agregar features sino cuidar los límites de los objetos. Cuando uno tiene asociaciones reales en el modelo, todo el resto del código (DAOs, servicios, controladores, tests) se vuelve más legible y más honesto.

También me llevo la convicción de que la consola sigue siendo una interfaz válida, sobre todo para prototipos. Hay una tentación inicial de ir directo a Swing o a una web, pero cuando uno se obliga a hacer todo en consola, aparecen oportunidades que con una GUI ya estarían resueltas (los menús ASCII, las tablas tabulares, los reportes en texto). Para un proyecto de gestión de práctica musical donde el usuario es un músico que abre la terminal dos veces por día para registrar una sesión, la consola es probablemente la mejor opción: rápida, repetible, y se puede usar con el teclado rápido sin tener que apuntar con el mouse.

Si en una iteración futura tuviera que hacer una versión web o de escritorio, el trabajo ya está facilitado: los DAOs son interfaces, los servicios están desacoplados, las excepciones son checked, las validaciones viven en Validador. Cualquier capa de presentación que respete esos contratos se enchufa sin modificar el resto. Pero por ahora, ReperTrack es una herramienta que funciona, y eso me alcanza.

Quedan pendientes para iteraciones futuras: notificaciones push cuando se dispara una alerta de mantenimiento, sincronización en la nube para poder usar el sistema desde el celular, y quizás un módulo de planificación semanal con metas configurables. Todo eso está en los requerimientos

candidatos del TP1 y son las líneas naturales de evolución del proyecto. Por ahora, con el prototipo en consola y 14/14 PASS, doy por cerrado el ciclo del AP1.

13. Referencias bibliográficas

Braude, E. (2003). Ingeniería de software: una perspectiva orientada a objetos. Alfaomega Grupo Editor.

Ericsson, K. A., Krampe, R. T., y Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363-406.

Institute of Electrical and Electronics Engineers. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE Standard No. 829).

Jacobson, I., Booch, G., y Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Kendall, K., y Kendall, J. (2011). Análisis y diseño de sistemas. Pearson Educación.

Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. E., y Abrutsky, M. A. (2012). Bases de datos. Alfaomega.

Rumbaugh, J., Jacobson, I., y Booch, G. (2006). El lenguaje unificado de modelado (2.ª ed.). Pearson-Addison-Wesley.

Saroka, R. (2002). Sistemas de información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison-Wesley.

Stair, R., y Reynolds, G. (2010). Principios de sistemas de información (9.ª ed.). Cengage Learning.

Anexo A. Diagramas UML del proyecto

A continuación se incluyen los cinco diagramas UML corregidos que cierran el ciclo del PUD para ReperTrack: dominio, clases de diseño, secuencia del CU006, despliegue local y entidad-relación con tipos SQL. Todos están generados con PlantUML (motor Smetana, sin dependencia de Graphviz) y exportados a PNG a 150 DPI.

A.1. Diagrama de Dominio

Modelo conceptual con Músico, Instrumento, ElementoDeEstudio (abstracta) y sus especializaciones Canción y Ejercicio, SesiónDePráctica y Estadística. Las asociaciones reflejan el modelo del TP1 corregido: Músico registra sesiones directamente, Instrumento agrupa elementos y se asocia a sesiones, y ElementoDeEstudio se aborda en sesiones.

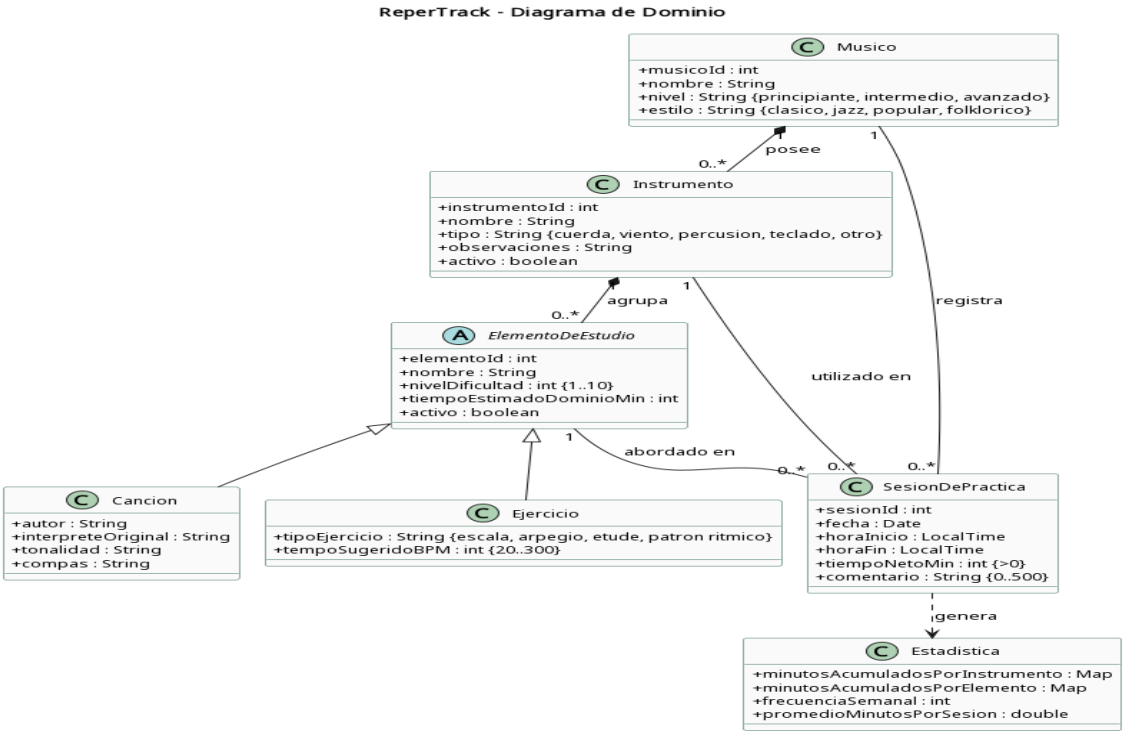


Figura 1. Diagrama de clases de dominio (corrección TP1).

A.2. Diagrama de Clases de Diseño

Vista física del modelo, agrupada por paquetes: modelo (con asociaciones explícitas Instrumento --Musico y ElementoDeEstudio --Instrumento), excepción, util, dao, service, controlador, estructuras y legacy. Se incluyen visibilidades, tipos Java y firmas de los métodos principales. Los nombres concretos de los controladores (MenuPrincipal, SesionController, etc.) coinciden con los usados en el diagrama de secuencia del Anexo A.3.

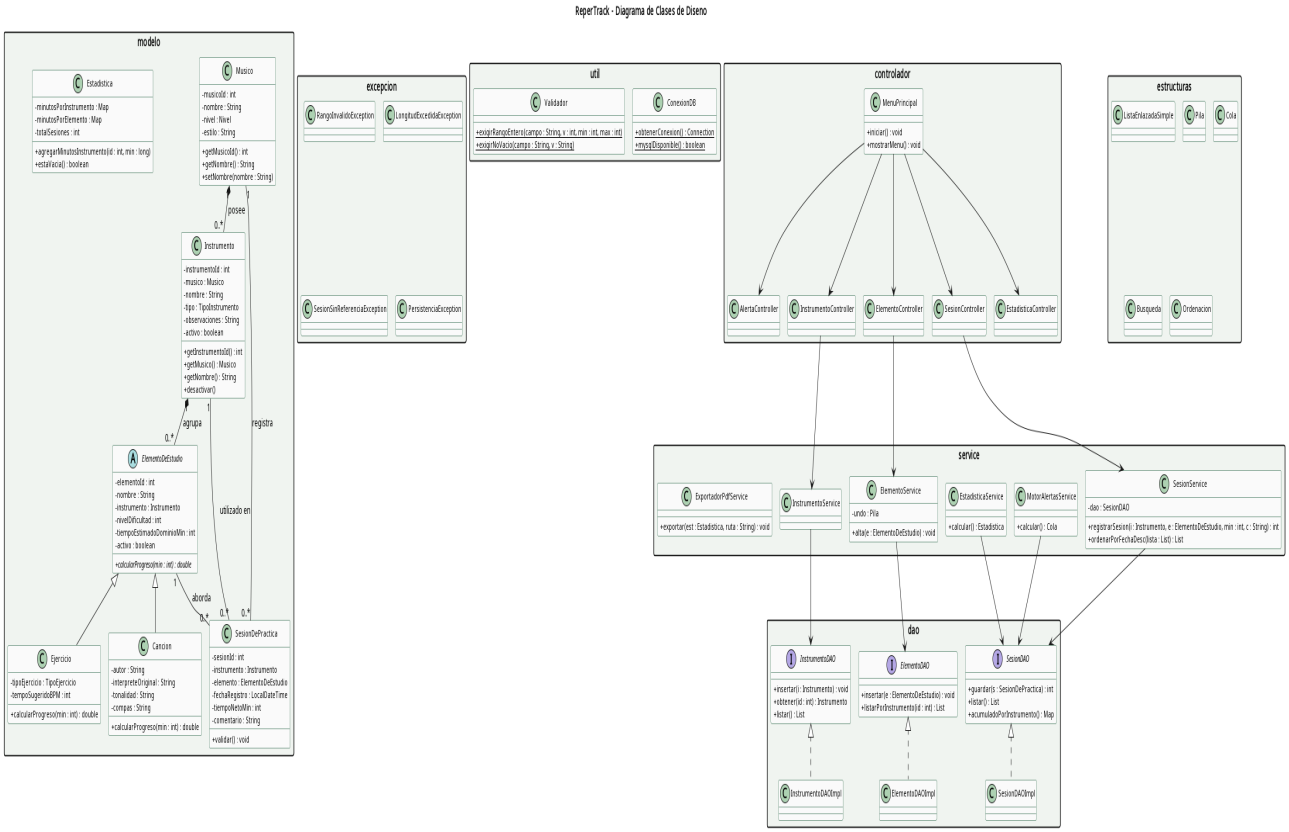


Figura 2. Diagrama de clases de diseño (corrección TP2).

A.3. Diagrama de Secuencia del CU006

Curso normal del caso de uso CU006 (Registrar Sesión de Práctica) utilizando exclusivamente las clases del diagrama de clases de diseño del Anexo A.2. La rama de transacción cubre el patrón `setAutoCommit(false) / commit()`; la nota final recuerda las reglas RN-02, RN-04 y RN-05 aplicadas en este flujo.

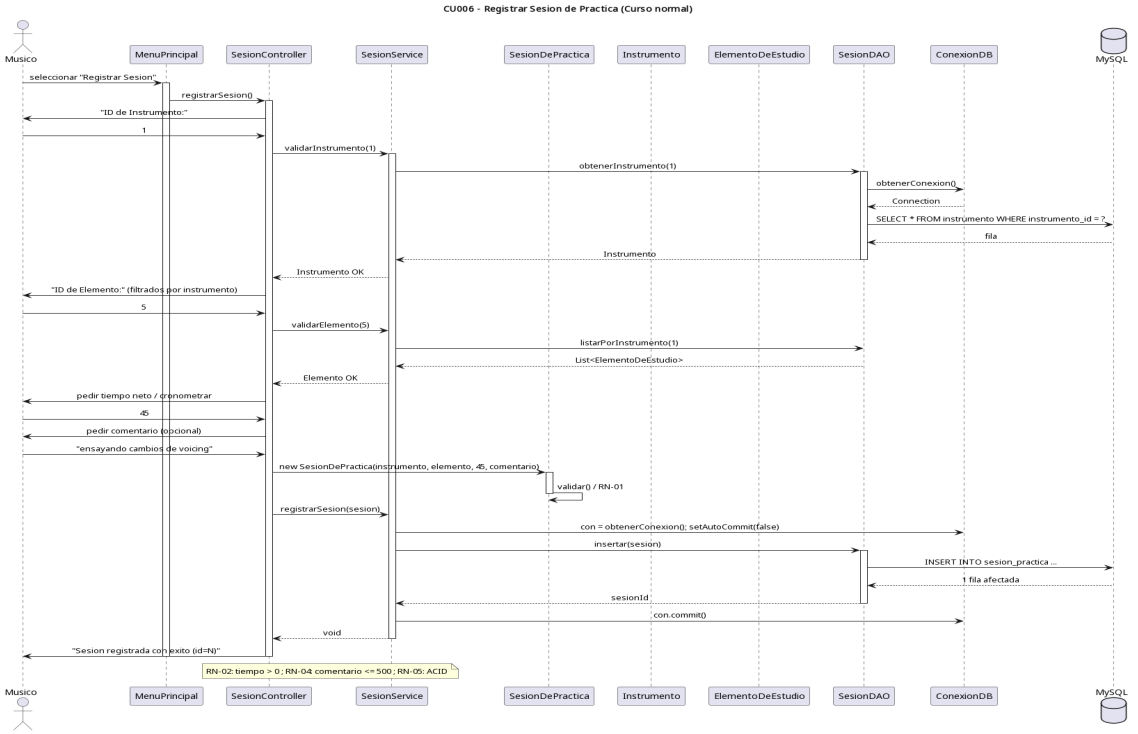


Figura 3. Diagrama de secuencia del CU006 (corrección TP2).

A.4. Diagrama de Despliegue Local

Despliegue monousuario: la PC corre la aplicación de consola sobre JRE 11 LTS y conversa con un MySQL 8.0 local mediante mysql-connector-j sobre TCP/IP loopback :3306. Sin servidor web, sin Apache/Tomcat/HTTP/XAMPP.

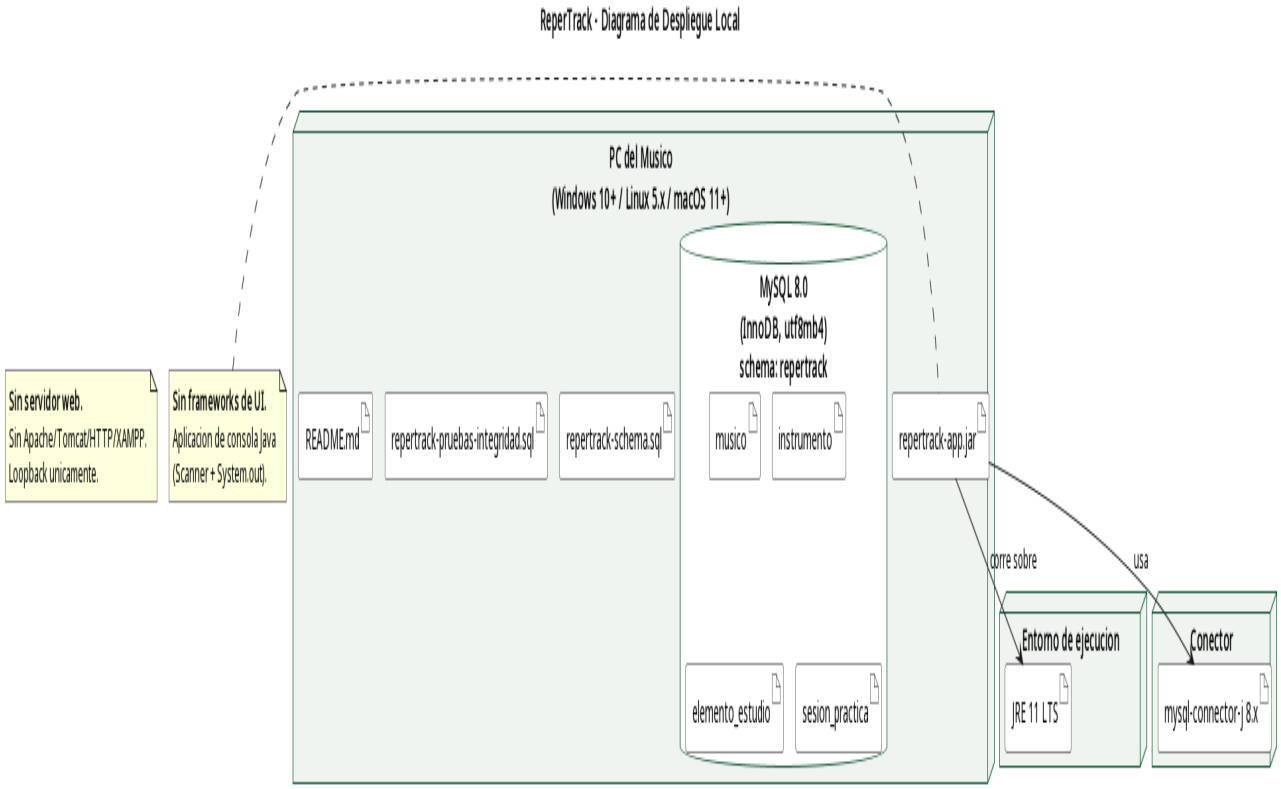


Figura 4. Diagrama de despliegue local (corrección TP2).

A.5. Diagrama Entidad-Relación (MySQL 8)

Esquema relacional con tipos SQL explícitos por atributo: claves primarias subrogadas, claves foráneas con `ON DELETE RESTRICT`, restricciones `UNIQUE`, `CHECK` enforcement para cada regla de negocio, e índices compuestos `idx_sesion_inst_ts` y `idx_sesion_elem_ts` que aceleran Q2..Q5 del informe. Single-table inheritance en elemento_estudio con la columna `categoria` distinguiendo Canción de Ejercicio.

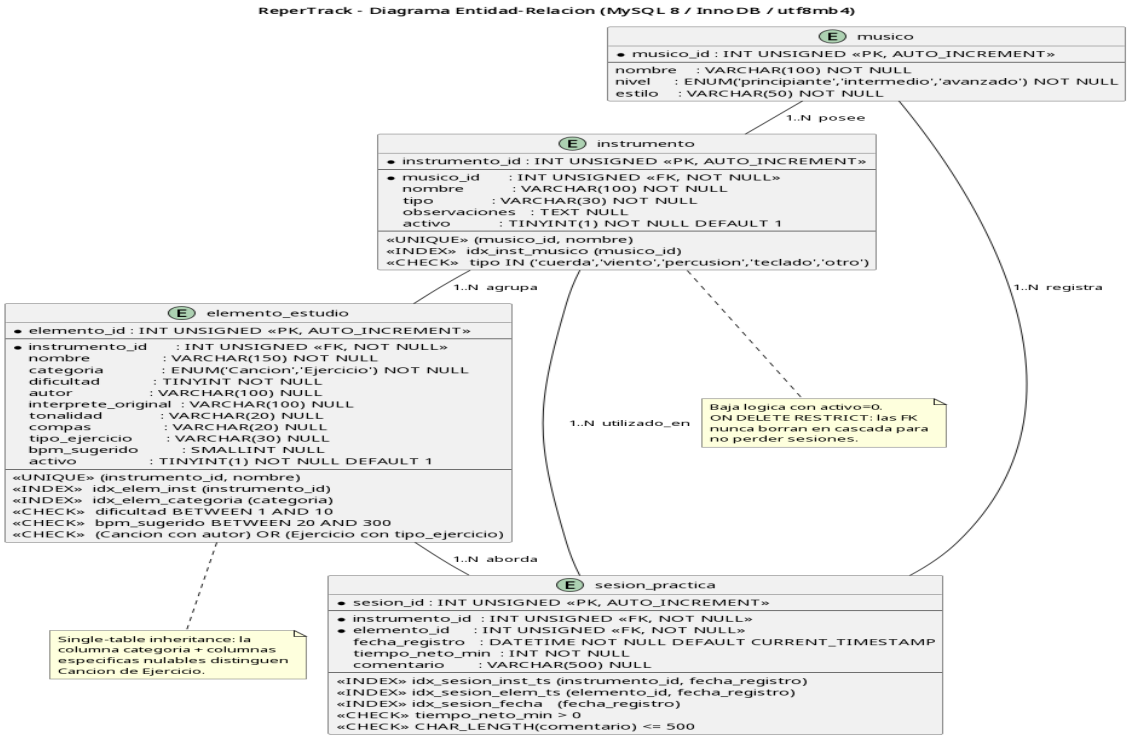


Figura 5. Diagrama Entidad-Relación con tipos SQL explícitos (corrección TP2).